Software Engineering

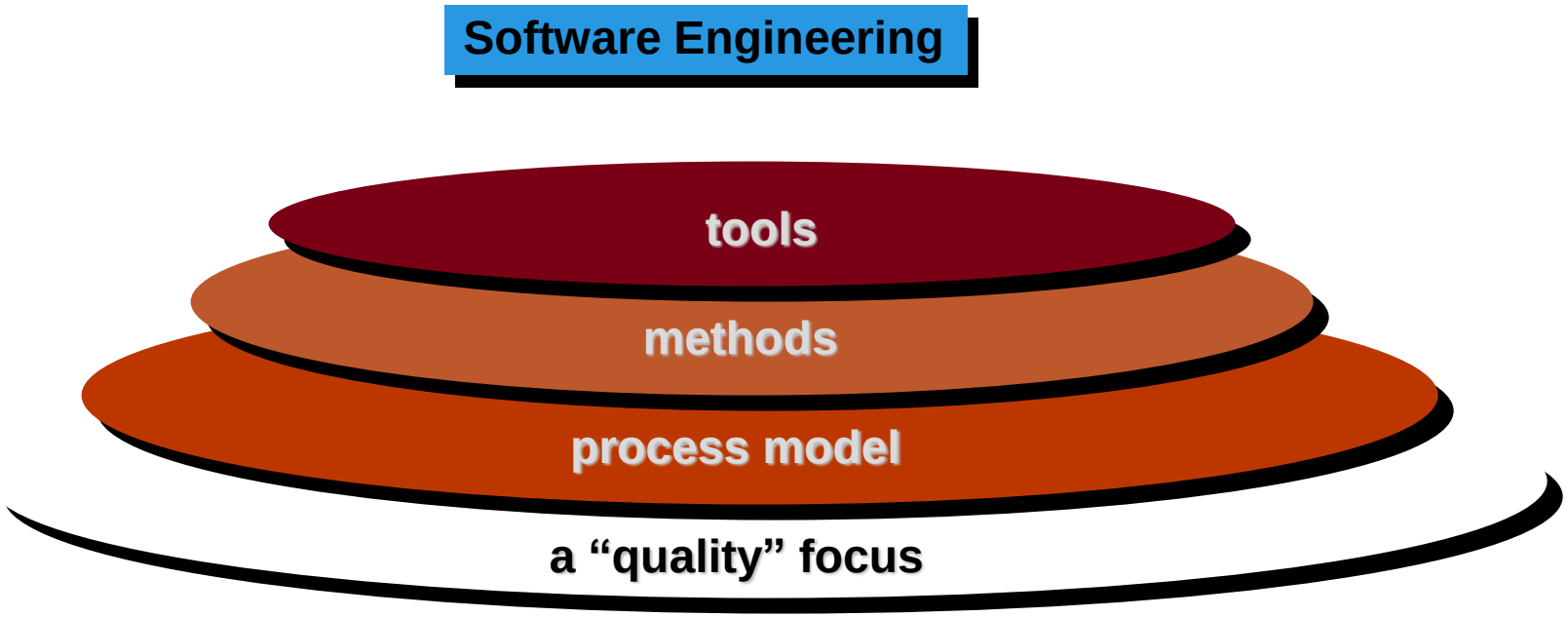
Chapter 2- (The Process)
Lecture 02

Farjana Parvin
Lecturer
Dept. of CSE, RUET

Process, Methods, and Tools

- Software engineering is a layered technology.

A Layered Technology



Process, Methods, and Tools

- **A quality focus**

- Any engineering approach (including software engineering) must rest on an organizational commitment to quality. It is the bedrock and focuses on organizational commitment to quality. It makes use of techniques like six sigma and TQM to achieve Quality.

Process, Methods, and Tools

- **Process**
  - The foundation for software engineering is the **process layer**. Software engineering process is the glue that holds the technology layers together and enables rational and timely development of computer software
  - Process defines the framework for a set key process areas that must be established for effective delivery of software engineering technology.

Process, Methods, and Tools

- **Methods**
  - Software engineering **methods** provide the technical "how to's" for building software.
  - Methods includes array of tasks that are, requirements analysis, design, program construction, testing, and maintenance.

- **Tools**
  - Software engineering **tools** provide automated or semi-automated support for the process and the methods.

Generic phases of software engineering process

The work associated with software engineering is associated with three generic phases.

- Definition phase
- Development phase
- Maintenance phase

Generic phases of software engineering process

- **Definition phase**

  •The software developer attempts to identify what information is to be processed, what function and performance are desired, what interfaces are to be established and what validation criteria are required to define a successful system.
  •Three major tasks will occur in some form: system or information engineering, software project planning, and requirements analysis

Generic phases of software engineering process

- **Development phase**

  •During development a software engineer attempts to define how data are to be structured, how function is to be implemented as a software architecture, how interfaces are to be characterized, how the design will be translated into a programming language and how testing is performed.
  •Three specific technical tasks should always occur: software design, code generation, and software testing.

Generic phases of software engineering process

- **Support phase**
  - Support phase focuses on changes. Four types of changes are,
    - Correction
    - Adaptation
    - Enhancement
    - Prevention

Umbrella Activities

- The phases described in generic view of software engineering are complemented by a number of umbrella activities.
- Typical activities include:
  - Software project tracking and control – allow the software team to assess progress against the project plan and take necessary action to maintain schedule.
  - Risk management – assesses risks that may effect the outcome of the product or the quality of the product.
  - Software quality assurance – defines and conducts the activities required to ensure software quality.

Umbrella Activities

- Formal technical reviews – assess software engineering work products in an effort to uncover and remove errors before they are propagated to the next action or activity.
- Software configuration management – manages the effects of change throughout the software process.
- Reusability management – defines criteria for work product reuse and establishes mechanisms to achieve reusable components.
- Work product preparation and production – encompasses the activities required to create work products such as models, documents, logs, forms, and lists.

Software process models

To solve actual problems in an industry setting, a software engineer or a team of engineers must incorporate a development strategy that encompasses the process, methods, and tools layers and the generic phases. This strategy is often referred to as a process model or a software engineering paradigm.

The SEI approach provides a measure of the global effectiveness of a company's software engineering practices and establishes five process maturity levels that are defined in the following manner:

Level 1: Initial. The software process is characterized as ad hoc and occasionally even chaotic. Few processes are defined, and success depends on individual effort.

Level 2: Repeatable. Basic project management processes are established to track cost, schedule, and functionality. The necessary process discipline is in place to repeat earlier successes on projects with similar applications.

Level 3: Defined. The software process for both management and engineering activities is documented, standardized, and integrated into an organization wide software process. All projects use a documented and approved version of the organization's process for developing and supporting software. This level includes all characteristics defined for level 2.

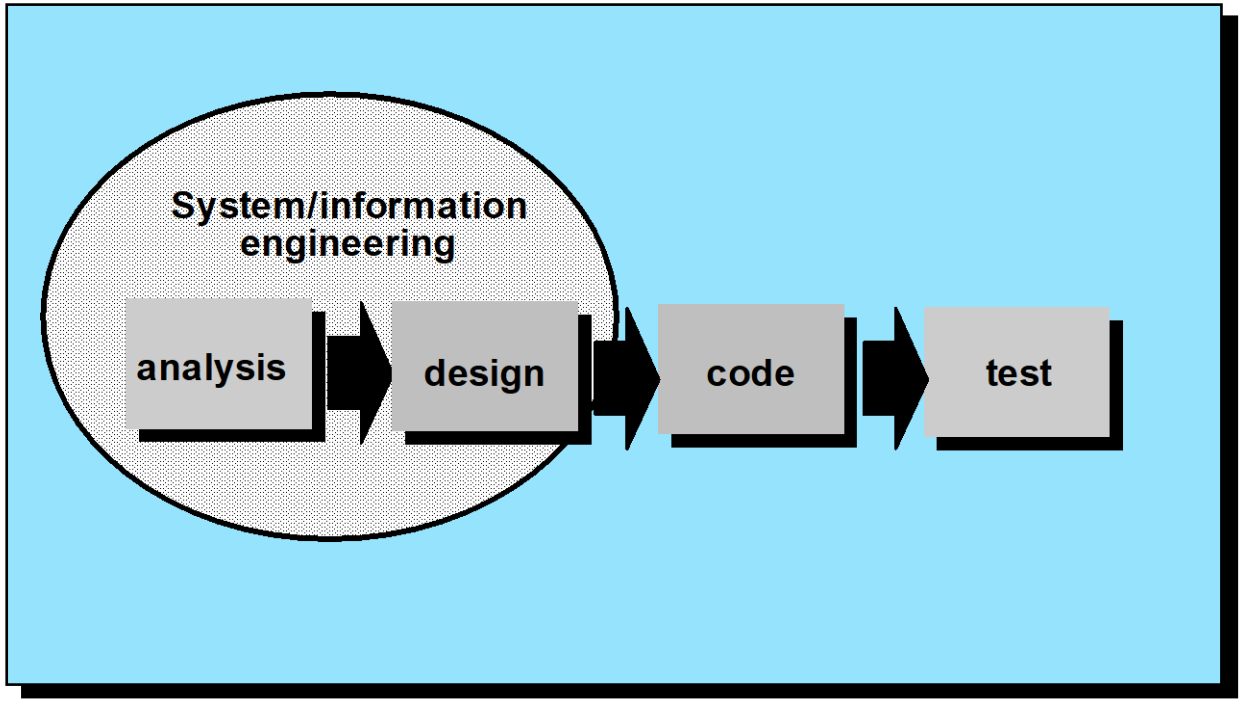
Level 4: Managed. Detailed measures of the software process and product quality are collected. Both the software process and products are quantitatively understood and controlled using detailed measures. This level includes all characteristics defined for level 3.

Level 5: Optimizing. Continuous process improvement is enabled by quantitative feedback from the process and from testing innovative ideas and technologies. This level includes all characteristics defined for level 4.

The SEI has associated key process areas (KPAs) with each of the maturity levels.
The KPAs describe those software engineering functions that must be present to satisfy good practice at a particular level. Each KPA is described by identifying the following characteristics:

- Goals—the overall objectives that the KPA must achieve.
- Commitments—requirements (imposed on the organization) that must be met to achieve the goals or provide proof of intent to comply with the goals.
- Abilities—those things that must be in place (organizationally and technically) to enable the organization to meet the commitments.
- Activities—the specific tasks required to achieve the KPA function.
- Methods for monitoring implementation—the manner in which the activities are monitored as they are put into place.
- Methods for verifying implementation—the manner in which proper practice for the KPA can be verified.

Software process models
The Linear Sequential Model



The Linear Sequential Model

- Some times called waterfall model
- It encompasses the following activities,
  - **System / information engineering and modeling**
    - System engineering and analysis encompasses requirements gathering at the system level
    - Information engineering encompasses requirements gathering at the strategic business level

The Linear Sequential Model

- Software requirements analysis

To understand the nature of the program(s) to be built, the software engineer ("analyst") must understand the information domain for the software, as well as required function, behavior, performance, and interface. Requirements for both the system and the software are documented and reviewed with the customer.

The Linear Sequential Model

- Design

- Software design is actually a multistep process that focuses on four distinct attributes of a program: data structure, software architecture, interface representations, and procedural (algorithmic) detail. The design process translates requirements into a representation of the software that can be assessed for quality before coding begins. Like requirements, the design is documented and becomes part of the software configuration

The Linear Sequential Model

- Code generation

- The design is translated into a machine readable form.

- Testing

- Testing the program to uncover errors and to ensure the defined input produce actual results.

The Linear Sequential Model

-Support.

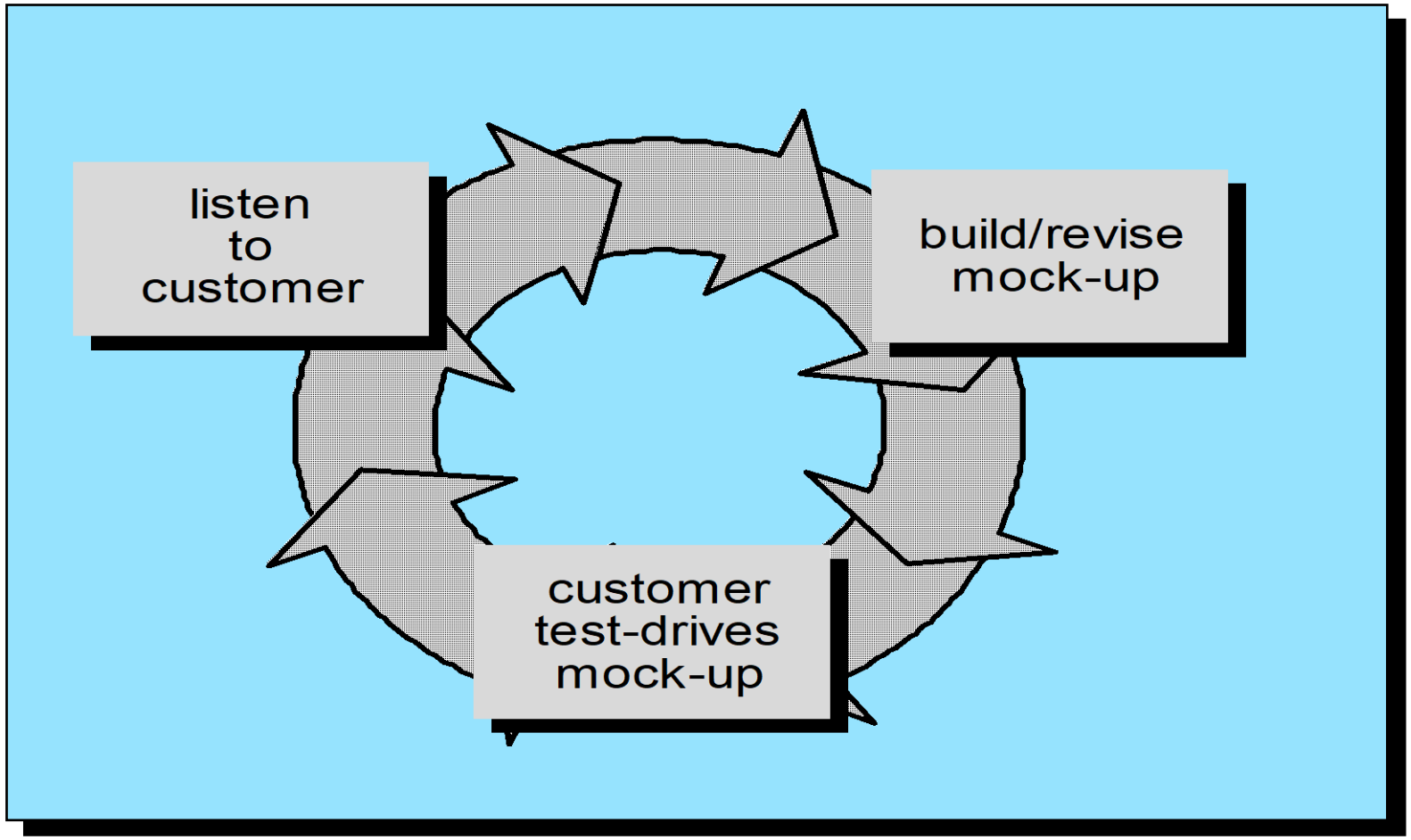
Software will undoubtedly undergo change after it is delivered to the customer. Change will occur because errors have been encountered, because the software must be adapted to accommodate changes in its external environment or because the customer requires functional or performance enhancements. Software support/maintenance reapplies each of the preceding phases to an existing program rather than a new one.

The Linear Sequential Model

The problems that are encountered when the Linear Sequential Model is applied are:

- Real projects rarely follow the sequential flow that the model proposes.
- It is difficult for the customer to state all requirements explicitly.
- The customer must have patients.
- Developers must wait for other members of the team to complete dependent tasks. This state is referred to as “blocking state”.

Prototyping



The Prototyping Model

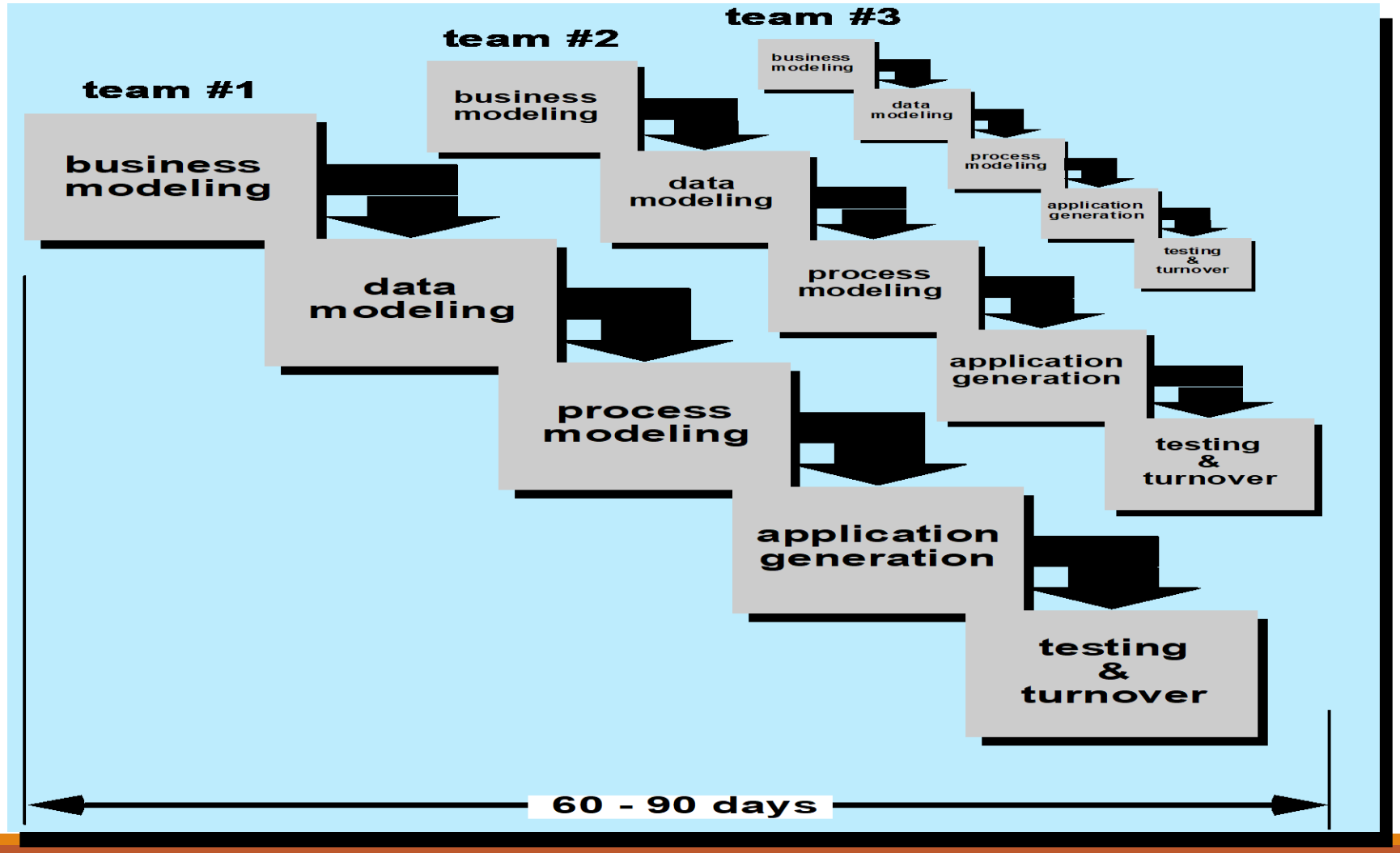
- The prototyping paradigm begins with requirements gathering.
- A quick design occurs. The quick design focuses on a representation of those aspects of the software that will be visible to the customer/user. The quick design leads to the construction of prototype.
- The prototype is evaluated by the customer/user and is used to refine requirements for the software to be developed.
- Iteration occurs as the prototype is tuned to satisfy the needs of the customer, at the same time enabling the developer to better understand what needs to be done

The Prototyping Model

Prototyping can be problematic for the following reasons:

• The customer sees what appears to be a working version of the software unaware that an effort is necessary to produce the working model.
• An inappropriate operating system or programming language may be used simply because it is available and known and an inappropriate algorithm may be used simply to demonstrate capability.

The RAD model



The RAD model

•Rapid Application Development (RAD) is a linear sequential software development process model that emphasizes an extremely short development cycle.

•If requirements are well understood and project scope is constrained, the RAD process enables a development team to create a “fully functional system” within very short time period

The RAD model

The RAD approach encompasses the following phases:

- **Business Modeling** – The information flow among business functions is modeled in a way that answers the following question: What information is generated? Who generates it?
- **Data Modeling** – The information flow is refined into a set of data objects that are needed to support the business.
- **Process Modeling** – The data objects defined in the data modeling phase are transformed to achieve the information flow necessary to implement a business function.
- **Application Generation** – The RAD process works to reuse existing program components or create reusable components.
- **Testing and Turnover** – New components must be tested and all interfaces must be fully exercised.

The RAD model

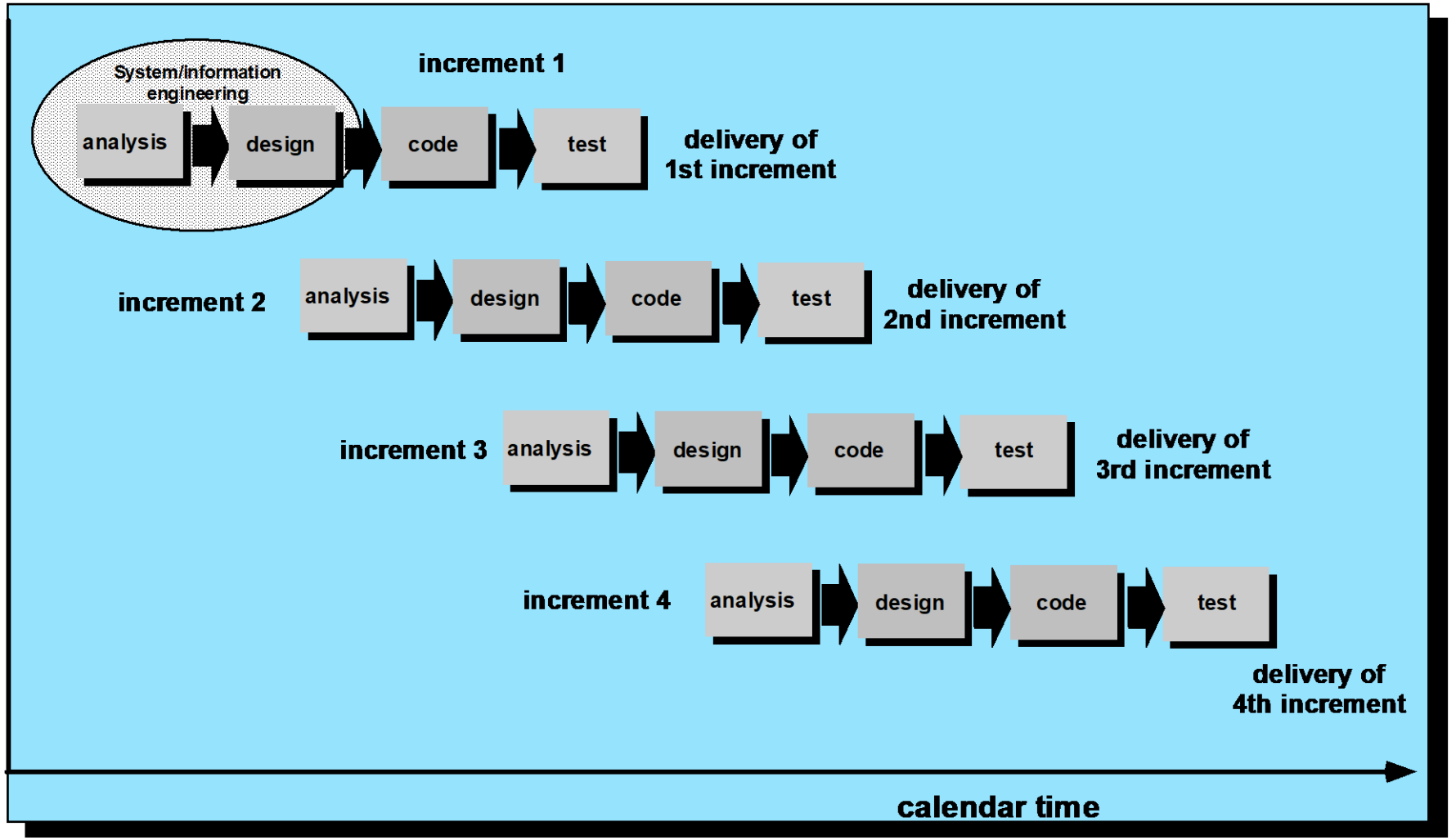
The drawbacks of RAD approach:

- For large, but scalable projects, RAD requires sufficient human resources to create the right number of RAD teams.
- RAD requires developers and customers who are committed to the rapid-fire activities necessary to complete a system in a much abbreviated time frame.
- If a system cannot be properly modularized, building the components for RAD will be problematic.
- RAD may not be appropriate when technical risks are high.

Evolutionary Software Process Models

• Evolutionary models are iterative.
• They are characterized in a manner that enables software engineers to develop increasingly more complete versions of the software.

The Incremental Model



The incremental model

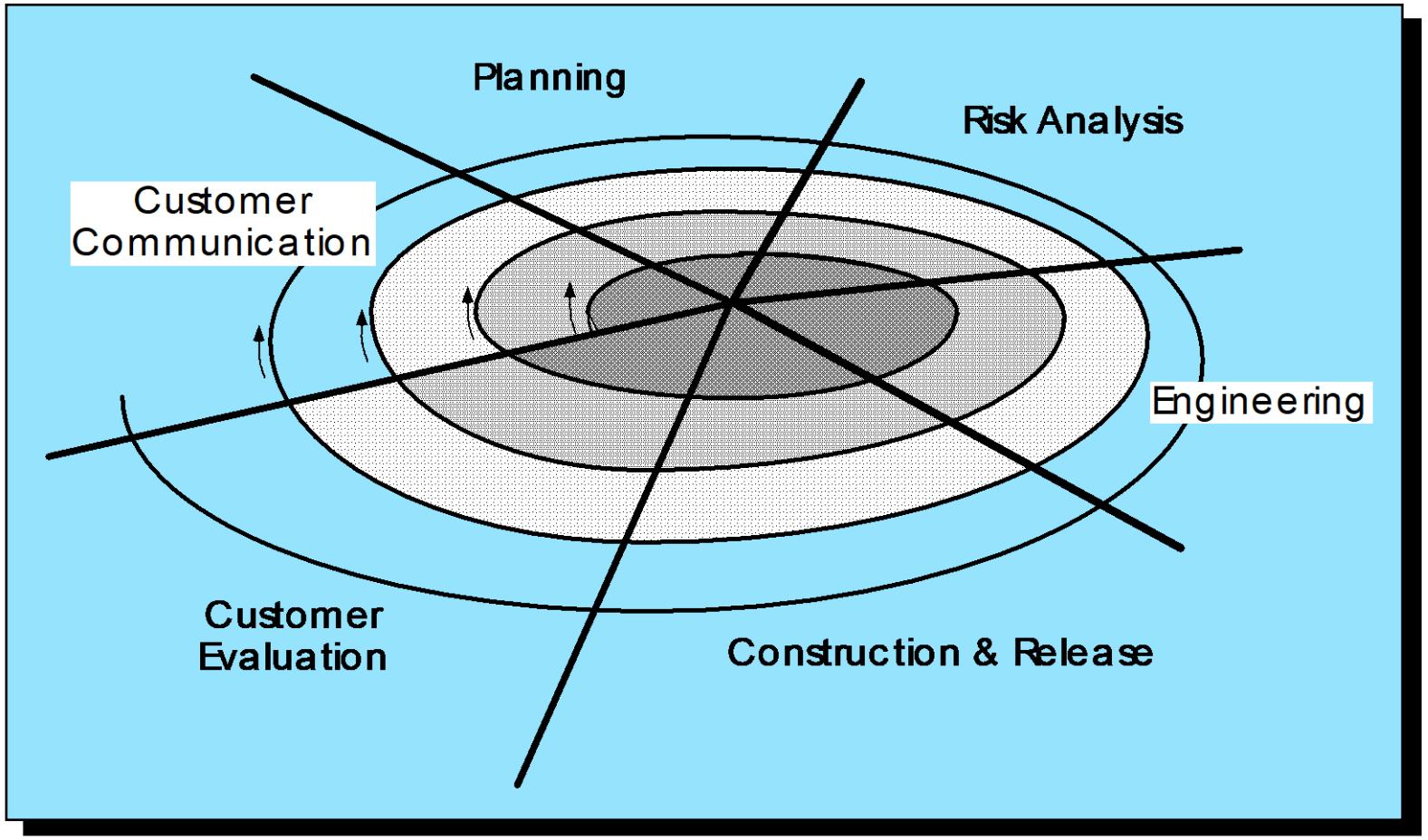
•Incremental model combines elements of the linear sequential model with prototyping
•In incremental model, the first increment is the core product (basic requirements are addresses, but many features remain undelivered)
•The core product is used by the customer, a plan is developed for the next increment
•The plan addresses the modification of the core product to better meet the user needs
•This process is repeated until the complete product is produced

The incremental model

Incremental model is useful for the following reasons:

- When staffing is unavailable for a complete implementation with fewer people.
- Increments can be planned to manage technical risks.

An Evolutionary (Spiral) Model



The spiral model

- In the spiral model, software is developed in a series of incremental releases.

- Spiral model contains six task regions,
  - Customer communication
  - Planning
  - Risk analysis
  - Engineering
  - Construction and release
  - Customer evaluation

- Spiral model is a realistic approach to the development of large scale systems and software

Still Other Process Models

- Component assembly model—**the process to apply when reuse is a development objective**
- Concurrent process model—**recognizes that different part of the project will be at different places in the process**
- Formal methods—**the process to apply when a mathematical specification is to be developed**
- Cleanroom software engineering—**emphasizes error detection *before* testing**

Thank You